

## Relatório - Matchmaking System

Alunos: Elian Miguel Ferreira Custódio, Luis Felipe Veloso Canha

### Algoritmos de Ordenação

#### *Insertion Sort*

O Insertion Sort percorre a lista da esquerda para a direita. A cada iteração  $i$ , o jogador  $players[i]$  é comparado com os elementos anteriores, deslocando eles para a direita até encontrar a posição correta. Em caso de empate no score, o desempate é resolvido no timestamp.

#### *Merge Sort*

O Merge Sort é implementado na forma recursiva, dividindo o array ao meio, e chamando a si mesmo para cada metade. Na volta, junta tudo com o *merge()*. A recursão utilizada divide o problema de tamanho  $n$  em dois subproblemas de tamanho  $n/2$ , até chegar ao caso base  $n = 1$ , trivialmente ordenado. Após cada merge, os arrays temporários *left* e *right* são deletados com *delete[]*, liberando memória. O critério de desempate em *merge()* é o mesmo do Insertion Sort.

### Formação de Grupos

A função *formGroup(groupSize, delta, n)* depende diretamente da lista estar previamente ordenada por score. Ela itera pelos jogadores e, a partir de cada índice  $i$ , tenta acumular *groupSize* jogadores consecutivos cujas pontuações diferem em no máximo *delta* do primeiro elemento do grupo. Como os jogadores estão em ordem crescente de score, as iterações acabam assim que a diferença excede *delta*, garantindo que nenhum possível jogador seja ignorado.

### Custo Computacional

*insert()*: Adiciona o jogador na próxima posição livre da lista e incrementa *size*. Custo  $O(1)$ .

*removePlayer()*: Após encontrar o elemento pelo ID, todos os elementos seguintes são empurrados para a esquerda. No melhor caso, quando o ID é o primeiro, a busca termina cedo, mas o deslocamento ainda é  $O(n)$ . No pior caso, quando o ID não existe na lista, toda a busca é percorrida. Custo  $O(n)$ .

*sortByScoreInsertion()*: O melhor caso ocorre quando a lista já está ordenada, mas o loop externo ainda itera  $n$  vezes. Custo  $O(n)$ . No pior caso, quando a lista está inversamente ordenada, cada elemento é comparado e deslocado com todos os anteriores. Custo  $O(n^2)$ .

*sortByScoreMerge()*: O custo é  $O(n \log(n))$  em todos os casos. A divisão gera uma árvore com profundidade  $\log(n)$ , e cada nível executa  $O(n)$  operações. O custo de espaço é  $O(n)$  por nível.

*formGroup()*: No melhor caso, o grupo é encontrado nos primeiros  $g$  elementos, custando  $O(g)$ . No pior caso, o algoritmo percorre até  $O(n)$  posições iniciais e para cada uma verifica até  $g$  jogadores seguintes, custando  $O(ng)$ .

*getWaitingPlayers()*: Aloca uma nova lista de tamanho  $n$  e copia todos os jogadores. Custo  $O(n)$ .

*printWaitingPlayers()*: Percorre toda a lista e exibe cada elemento. Custo  $O(n)$ .

## Comparação entre Insertion Sort e Merge Sort

Para gerar os testes, nós criamos uma função *newPlayers* que gera Players aleatórios, cujos scores são gerados por uma função *pseudoaleatory*, que retorna um valor inteiro entre 0 e 3000.

Por sua vez, para realizar as comparações, nós criamos duas funções (uma para o insertion sort e outra para o merge sort) que utilizam a biblioteca <chrono> para medir o respectivo tempo de execução, e uma função *performanceTests*, que calcula o tempo médio de 5 execuções de cada função de ordenação.

Por fim, nós criamos uma última função *runPerformanceTests*, que chama a função de performance para os valores descritos abaixo:

| $n$    | Tempo (Insertion Sort) | Tempo (Merge Sort) |
|--------|------------------------|--------------------|
| 1000   | 4.2 ms                 | 0 ms               |
| 5000   | 134.8 ms               | 2.2 ms             |
| 10000  | 494.8 ms               | 5 ms               |
| 25000  | 3049.4 ms              | 16.4 ms            |
| 50000  | 11815.8 ms             | 37.4 ms            |
| 100000 | 46208.8 ms             | 74.6 ms            |

Nós podemos perceber que, desde valores pequenos de  $n$ , até valores grandes, o Insertion Sort possui um tempo maior de execução do que Merge Sort, apresentando um crescimento muito rápido em relação a esta. Para  $n = 25000$  (e, claro, já para valores menores de  $n$ , por volta de 15000), o algoritmo de Insertion Sort passa a ordenar os jogadores com um tempo na ordem dos segundos, enquanto o Merge Sort faz a ordenação sempre na ordem dos milissegundos, para qualquer valor de  $n$  até 100000.

Este resultado é, de fato, esperado, dado que o algoritmo de Insertion Sort possui uma complexidade temporal  $O(n^2)$ , enquanto o algoritmo de Merge Sort possui complexidade temporal  $O(n \log(n))$ , que corresponde a uma família de funções que cresce muito mais devagar em relação a qualquer função quadrática.

Por sua vez, conhecida uma precisão base de cada função para um certo  $n$ , digamos, 5000, podemos estimar matematicamente qual será o tempo que cada função irá gastar para outros valores de  $n$  (neste caso, por exemplo, tomando  $n/5000$ , substituindo na respectiva função de complexidade, e multiplicando o tempo gasto por um algoritmo para  $n = 5000$  pelo respectivo valor da função) e mostrar que tais estimativas são bem próximas do tempo realmente observado, o que evidencia a confiabilidade dos dados coletados.